АРХИТЕКТУРА ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ 2026
Задание №1

1. **ФИО**: Лазарь Владислав Игоревич.

2. **Группа**: *Б01-207а.*

3. **Вариант**: компаратор двух *2*-битных входов с выходами LEQ, EQ, G.

4. **Почта**: *lazar.v@phystech.edu.*

5. **Назначение схемы. Краткое описание принципов ее работы, ее входов/выходов.**

   (a) *Компараторы* определяют, являются ли два двоичных числа равными или одно из них больше/меньше другого. Компаратор получает два N-разрядных двоичных числа A и B. Существует два типа компараторов:

   - *компаратор* равенства выдает выходной сигнал, показывая, равны ли A и B;
   - *компаратор* величины выдает один и более выходных сигналов, показывая отношение величин A и B.

   По заданию необходимо реализовать компаратор величины, который вычисляет $A-B$ и анализирует знак (самый старший разряд) результата. Если результат отрицательный (самый старший разряд $= 1$), то $A < B$. В противном случае $A \geqslant B$.

   Заполним таблицу истинности для интересующего нас компаратора:

Входы				Выходы		
A_1	A_0	B_1	B_0	$A \leqslant B$	$A = B$	$A > B$
0	0	0	0	1	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	1	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	1	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	1	1	0

(b) **Оптимизация выходов в логическом приближении.**

Рассмотрим на примере выхода $\big(A > B\big) \equiv G$ как можно построить его логическую схему оптимальным образом. Заметим, что в столбце для G из *16* строк есть только *6* единиц — это подталкивает нас к выбору совершенной дизъюнктивной нормальной формы. Тогда имеем для G:

$$G = \overline{A_1}A_0\overline{B_1}\ \overline{B_0} + A_1\overline{A_0}\ \overline{B_1}\ \overline{B_0} + A_1\overline{A_0}\ \overline{B_1}B_0 + A_1A_0\overline{B_1}\ \overline{B_0} + A_1A_0\overline{B_1}B_0 + A_1A_0B_1\overline{B_0}.$$

Теперь можно уже традиционным для СДНФ способом собрать эту схему. Однако, она будет громоздкой, её можно упростить — для этого воспользуемся картой Карно:

B_1B_0 \ A_1A_0	00	01	11	10
00	0	1	1	1
01	0	0	1	1
11	0	0	0	0
10	0	0	1	0

Теперь для G имеем:

$$\boxed{G = A_1\overline{B_1} + A_0\overline{B_1}\ \overline{B_0} + A_1A_0\overline{B_0}}. \tag{1}$$

И, вообще говоря, это можно было бы получить из других соображений.

- Если $A_1 > B_1$, то мы сразу можем сказать, что $A > B$.
- Если $A_1 = B_1$, и при этом $A_0 > B_0$, то $A > B$.

Ровно это и записано в выражении (1).

Аналогично поступим для $\big(A = B\big) \equiv E$. Заметим, что в столбце для E из *16* строк есть только *4* единицы — снова используем совершенную дизъюнктивную нормальную форму. Тогда имеем для E:

$$E = \overline{A_1}\ \overline{A_0}\ \overline{B_1}\ \overline{B_0} + \overline{A_1}A_0\overline{B_1}B_0 + A_1\overline{A_0}B_1\overline{B_0} + A_1A_0B_1B_0. \tag{2}$$

Это выражение, очевидно, никак с помощью карт Карно нельзя упростить. Однако, это не значит, что никаким другим способом упростить нельзя. Воспользуемся теоремами логики явно, упростив выражение (2):

$$E = \big(A_1B_1 + \overline{A_1}\ \overline{B_1}\big)\cdot\big(A_0B_0 + \overline{A_0}\ \overline{B_0}\big) = \big(A_1 \odot B_1\big)\cdot\big(A_0 \odot B_0\big) = \boxed{\overline{\big(A_1 \oplus B_1\big)} \cdot \overline{\big(A_0 \oplus B_0\big)} = E}.$$

А для $(A \leqslant B) \equiv LEQ$ всё совсем просто: $\boxed{LEQ = \overline{G}}$, в чём можно убедиться логически или из таблицы истинности.

(c) **Оптимизация выходов в физическом приближении.**

В прошлом пункте были получены оптимальные логические выражения для выходов компаратора без учёта базиса вентилей:

- $G = A_1\overline{B_1} + A_0\overline{B_1}\ \overline{B_0} + A_1A_0\overline{B_0}$;
- $E = \overline{\left(A_1 \oplus B_1\right)} \cdot \overline{\left(A_0 \oplus B_0\right)}$;
- $LEQ = \overline{G}$.

Для реализации схем этих логичеких выражений, необходимо их привести к виду, в котором они имели бы оптимальное количество вентилей, ведь согласно условию разрешается использовать только **NOT**, **NOR**, **NAND**. Рассмотрим G:

$$A_1\overline{B_1}+A_0\overline{B_1}\ \overline{B_0}+A_1A_0\overline{B_0} = \overline{\overline{A_1\overline{B_1} + A_0\overline{B_1}\ \overline{B_0} + A_1A_0\overline{B_0}}} = \underbrace{\underbrace{\overline{\left(A_1\overline{B_1}\right)}}_{\mathbf{NAND}} \cdot \underbrace{\overline{\left(A_0\overline{B_1}\ \overline{B_0}\right)}}_{\mathbf{NAND}^3} \cdot \underbrace{\overline{\left(A_1A_0\ \overline{B_0}\right)}}_{\mathbf{NAND}^3}}_{\mathbf{NAND}^3}.$$

Причём, вот как можно наиболее эффективно из имеющихся вентилей собрать $\mathbf{NAND}^3$:

$$\mathbf{NAND}^3(A,\, B,\, C) = \mathbf{NAND}\Big(\mathbf{NOR}\Big(\mathbf{NOT}(A),\, \mathbf{NOT}(B)\Big),\, C\Big).$$

Хотя, может показать, что вот такая реализация лучше:

$$\mathbf{NAND}^3(A,\, B,\, C) = \mathbf{NAND}\Big(\mathbf{NOT}\Big(\mathbf{NAND}(A,\, B)\Big),\, C\Big).$$

Но это не так. Дальнешее исследование поведения выхода E покажет, что нам всё равно нужно иметь все *4* инвертированных входа. И раз нам они и так будут нужны для другой части компаратора, то нет смысла "экономить" на элементах **NOT**, потому что бездумное использование второй реализации наоборот — только увеличит использование **NOT**. Будем использовать ту или иную реализацию $\mathbf{NAND}^3$ в зависимости от ситуации. Итак, для $\mathbf{NAND}^3$ нужны $\Big[2\text{x}\mathbf{NOT},\, 1\text{x}\mathbf{NOR},\, 1\text{x}\mathbf{NAND}\Big]$.

Для всей схемы G потребовались бы $\Big[2\text{x}\mathbf{NOT},\, 1\text{x}\mathbf{NAND},\, 3\text{x}\mathbf{NAND}^3\Big]$. Если считать, что у нас есть все инвертированные входы по умолчанию, то в пересчёте на базисные вентили нам понадобились бы $\Big[3\text{x}\mathbf{NOR},\, 4\text{x}\mathbf{NAND}\Big]$.

И складывается ощущение, как будто можно лучше. Запишем G в другом виде:

$$G = A_1\overline{B_1} + A_0\overline{B_0} \cdot \left(A_1 + \overline{B_1}\right) = \overline{\overline{A_1\overline{B_1} + A_0\overline{B_0} \cdot \overline{\overline{\left(A_1 + \overline{B_1}\right)}}}} = \underbrace{\underbrace{\overline{\left(A_1\overline{B_1}\right)}}_{\mathbf{NAND}} \cdot \underbrace{\overline{\left(A_0\overline{B_0} \cdot \underbrace{\overline{\overline{A_1}B_1}}_{\mathbf{NAND}}\right)}}_{\mathbf{NAND}^3}}_{\mathbf{NAND}}.$$

А вот теперь для G нам нужны $\Big[3\text{x}\mathbf{NAND},\, 1\text{x}\mathbf{NAND}^3\Big]$. Или в пересчёте на базисные вентили — $\Big[1\text{x}\mathbf{NOR},\, 4\text{x}\mathbf{NAND}\Big]$, что заметно лучше прошлой имплементации.

Теперь рассмотрим выход E:

$$E = \overline{\left(A_1 \oplus B_1\right)} \cdot \overline{\left(A_0 \oplus B_0\right)} = \overline{\left(A_1 \oplus B_1\right) + \left(A_0 \oplus B_0\right)} = \overline{\left(\overline{A_1}B_1 + A_1\overline{B_1}\right) + \left(\overline{A_0}B_0 + A_0\overline{B_0}\right)}.$$

Немного шаманства (несколько часов перебора наилучшего варианта):

$$E = \overline{\overline{\overline{\overline{\left(\overline{A_1}B_1 + A_1\overline{B_1}\right)} + \overline{\left(\overline{A_0}B_0 + A_0\overline{B_0}\right)}}}} = \underbrace{\overline{\underbrace{\overline{\underbrace{\left(\overline{A_1}B_1\right)}_{\textbf{NAND}} \cdot \underbrace{\left(A_1\overline{B_1}\right)}_{\textbf{NAND}}}}_{\textbf{NAND}} + \underbrace{\overline{\underbrace{\left(\overline{A_0}B_0\right)}_{\textbf{NAND}} \cdot \underbrace{\left(A_0\overline{B_0}\right)}_{\textbf{NAND}}}}_{\textbf{NAND}}}}_{\textbf{NOR}}.$$

Значит, для E нам нужны $\Big[$1x**NOR**, 6x**NAND**$\Big]$.

Для LEQ всё снова просто. Так как $LEQ = \overline{G}$, то для LEQ нам нужен только $\Big[$1x**NOT**$\Big]$. И в этом месте мы должны выбрать, что для нас будет лучше — с одной стороны, добавленный **NOT** к выходу G это минимум затрат и с точки зрения транзисторной логики, и места на кристалле, и наводок, но с другой стороны это увеличит критический путь на *1*. Но я буду руководствоваться предположением, что место на кристалле, наводки и количество логических элементов для меня более приоритетны.

Итак, резюмируем финальный вид логических схем и количество базисных вентилей для этих схем при рассмотрении G и E в отрыве друг от друга:

- $G = \underbrace{\overline{\underbrace{\left(A_1\overline{B_1}\right)}_{\textbf{NAND}} \cdot \underbrace{\overline{\left(A_0\overline{B_0} \cdot \underbrace{\overline{\overline{A_1}B_1}}_{\textbf{NAND}}\right)}}_{\textbf{NAND}^3}}}_{\textbf{NAND}}$ — $\Big[$1x**NOR**, 4x**NAND**$\Big]$.
- $E = \underbrace{\overline{\underbrace{\overline{\underbrace{\left(\overline{A_1}B_1\right)}_{\textbf{NAND}} \cdot \underbrace{\left(A_1\overline{B_1}\right)}_{\textbf{NAND}}}}_{\textbf{NAND}} + \underbrace{\overline{\underbrace{\left(\overline{A_0}B_0\right)}_{\textbf{NAND}} \cdot \underbrace{\left(A_0\overline{B_0}\right)}_{\textbf{NAND}}}}_{\textbf{NAND}}}}_{\textbf{NOR}}$ — $\Big[$1x**NOR**, 6x**NAND**$\Big]$.
- $LEQ = \overline{G}$ — $\Big[$1x**NOT**$\Big]$.

(d) **Логические заимствования.**

И даже это не всё, что мы можем сделать. Эта реализация реально была бы оптимальной, если бы мы рассматривали выход E в "вакууме". Но ведь у нас уже построен выход G. Оттуда можно проводами вытащить логические выражения $\overline{A_1\overline{B_1}}$, $\overline{\overline{A_1}B_1}$, тем самым снизив количество **NAND** во всей схеме на *2* штуки.

И наоборот. Из схемы для E можно вытащить для схемы G выражение $\overline{A_0\overline{B_0}}$.

6. **Реализация в Logisim. Значения выходов схемы при разных значениях входа.**

   Исходя из всех улучшений для компаратора, предложенных выше, реализация в **Logisim** выглядит следующим образом (покажем сразу *4* варианта схемы с разными входами. Все возможные варианты можно найти в директории `./testing_comparator/`):

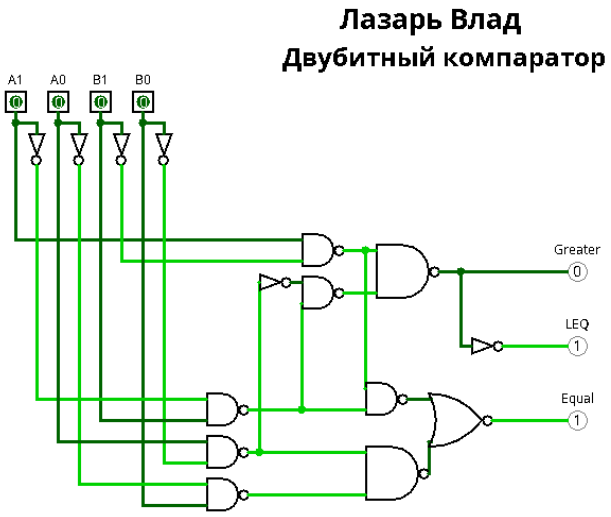


Рис. 1: $A = 00 \leqslant 00 = B$

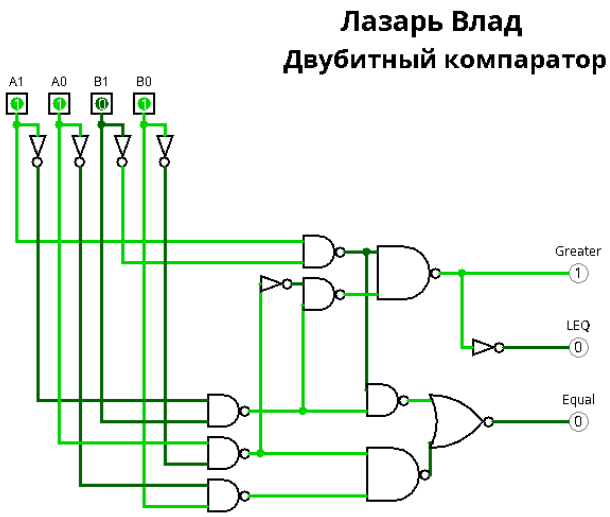


Рис. 2: $A = 11 > 01 = B$

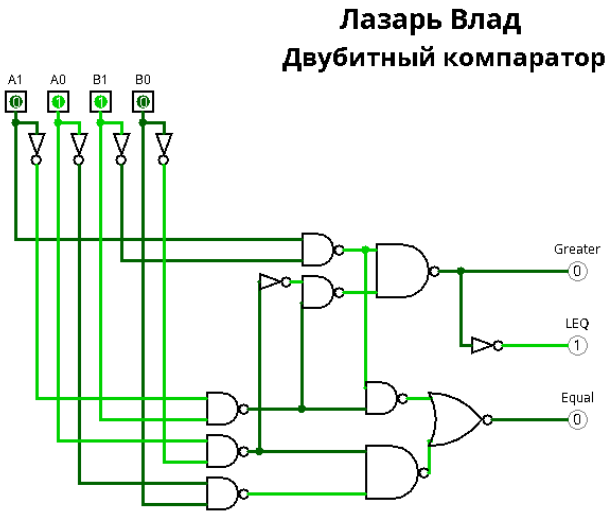


Рис. 3: $A = 01 \leqslant 10 = B$

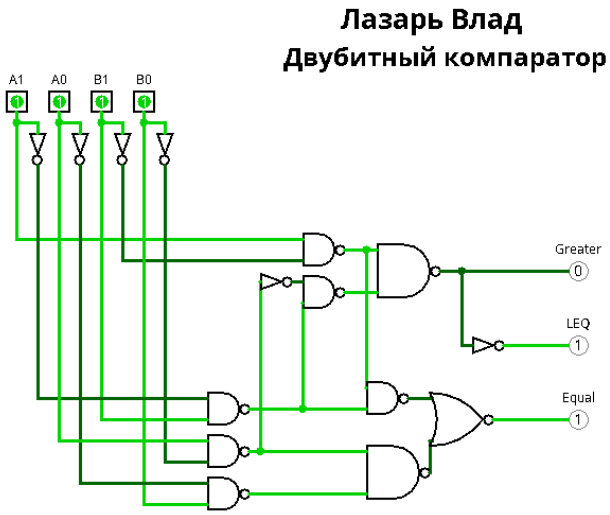
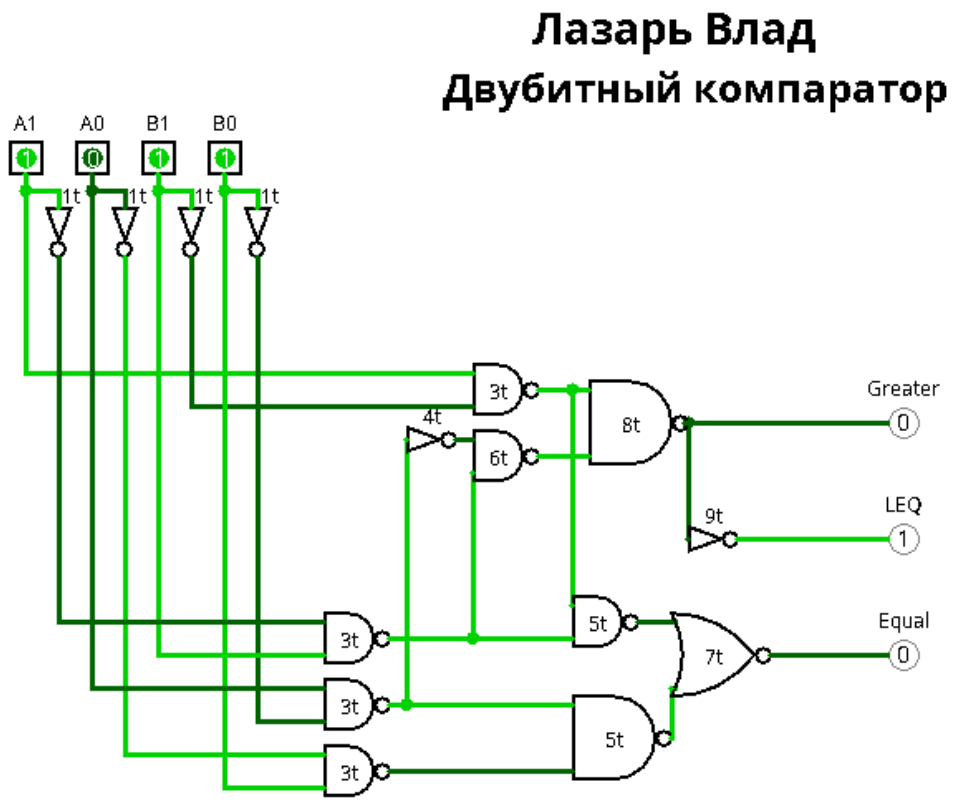


Рис. 4: $A = 11 \leqslant 11 = B$

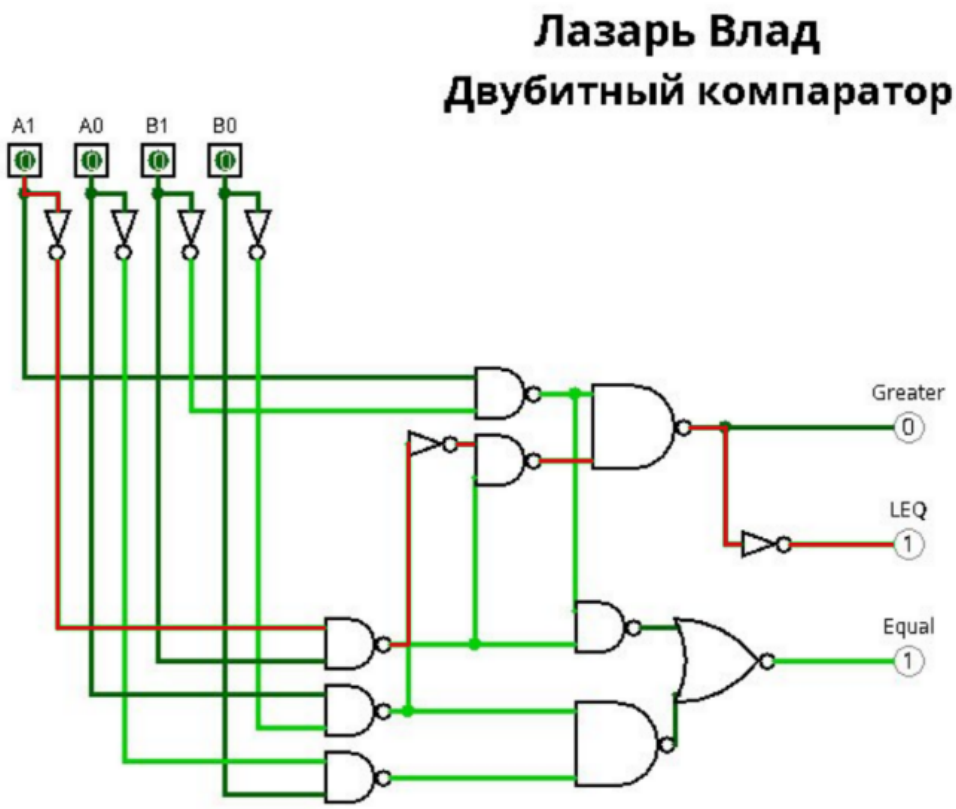
7. **На всех фото нанесены ФИО и название схемы.**

8. **Показать критический путь в схеме красной линией.**

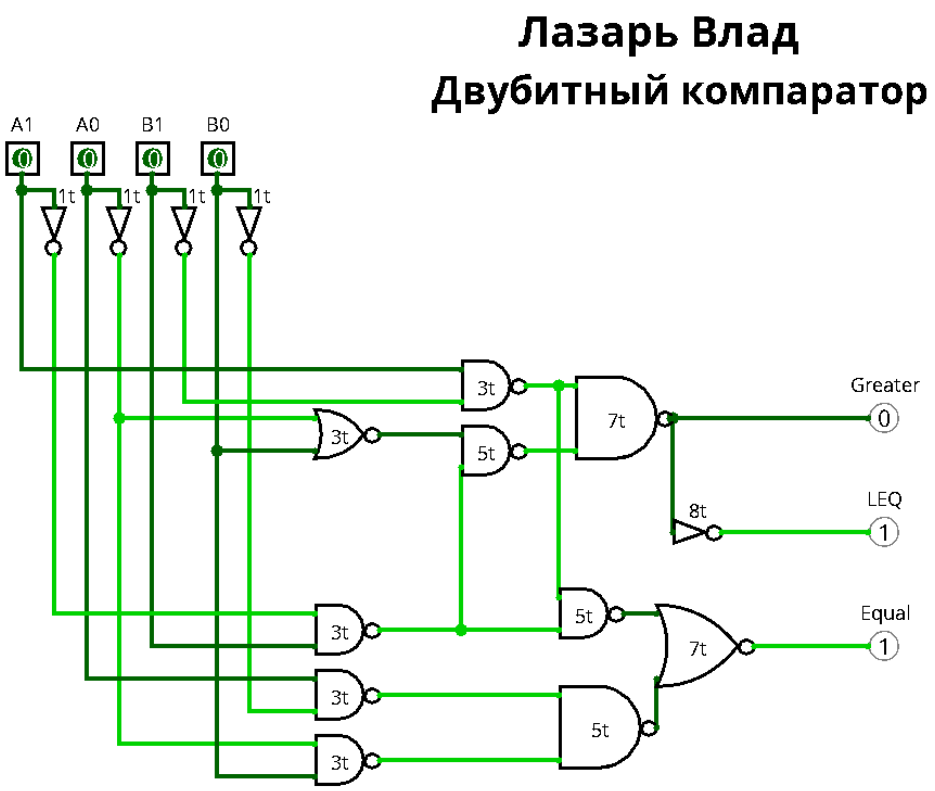
Сперва изобразим задержки на всех вентилях:



Теперь изобразим критический путь, который численно равен *9*.



При желании, критический путь можно снизить до *8*. Но тогда мы увеличим наводки в схеме и общее число транзисторов на *2*:



9. **Рассчитать примерное число транзисторов в схеме. При подсчете транзисторов PROBE и PIN не учитываются.**

   На всю схему получилось $\Big[6\text{x}\mathbf{NOT},\ 1\text{x}\mathbf{NOR},\ 8\text{x}\mathbf{NAND}\Big]$.

   - $\Big[1\text{x}\mathbf{NOT}\Big] \equiv \Big[1\text{x}\mathbf{PMOS},\ 1\text{x}\mathbf{NMOS}\Big]$.
   - $\Big[1\text{x}\mathbf{NOR}\Big] \equiv \Big[2\text{x}\mathbf{PMOS},\ 2\text{x}\mathbf{NMOS}\Big]$.
   - $\Big[1\text{x}\mathbf{NAND}\Big] \equiv \Big[2\text{x}\mathbf{PMOS},\ 2\text{x}\mathbf{NMOS}\Big]$.

   Тогда в пересчёте на транзисторы во всей схеме вышло $\Big[24\text{x}\mathbf{PMOS},\ 24\text{x}\mathbf{NMOS}\Big]$ с критическим путём, численно равным 9 $\Big($или же $\Big[25\text{x}\mathbf{PMOS},\ 25\text{x}\mathbf{NMOS}\Big]$ с критическим путём, численно равным 8. В зависимости от того, что для нас приоритетнее$\Big)$.

   Итак, всего в схеме *48* транзисторов. Наверняка можно было бы лучше, если разрабатывать компаратор на уровне транзисторной логики. Также можно было бы на этапе оптимизации выходов в логическом приближении воспользоваться методом Куайна – Мак-Класки.